A

자료실 · 실습용

실습 키트 (복사해서 바로 쓰기)

강의에 나온 지시문과 소스를 순서대로 모았습니다. 상자마다 복사 버튼이 있습니다.

시작 시작

강의 중 나온 모든 지시문(프롬프트)과 소스를 순서대로 모았습니다. 회색 상자마다 **복사** 버튼이 있습니다. 옵시디언에서는 상자 오른쪽 위 복사 아이콘을 누릅니다.

실습 폴더 받기

함께 받은 `agent-lab.zip` 을 풀면 아래 4~6강 파일(CLAUDE.md, 설정, 훅, 스킬, 서브에이전트, 가짜 데이터)이 모두 들어 있습니다. 압축을 풀었다면 0단계의 "파일 직접 만들기"는 건너뜁니다.

실습 안내

모든 데이터는 가짜입니다. 실제 개인정보·내부 문서는 넣지 않습니다.

실습 1. 대화형 AI 실습 (1~3강, 설치 불필요)

Claude, ChatGPT, Gemini 중 아무 서비스에서 합니다.

1강. AI 역사 사실 확인

```text
AI의 겨울이 왜 왔는지 소방관에게 비유해서 3줄로 설명해줘. 연도와 인물이 나오면 함께 적어줘.
```

답에 나온 연도·인물을 1강 표와 대조합니다.

2강. 같은 일, 두 가지 소원

먼저 막연한 요청:

```text
소방 안전교육 자료 만들어줘
```

새 대화에서 구체적인 요청:

```text
초등 3학년 대상, 10분짜리 화재 대피 안전교육 대본을 써줘. 아이에게 던질 질문 2개를 포함하고, 쉬
```

2강. 사실 확인 요청

```text
방금 답에서 사실 확인이 필요한 문장만 골라 표로 정리하고, 확인할 수 있는 공식 출처를 함께 적어줘
```

3강. 내가 쓰는 모델 확인

```text
너는 어떤 회사의 어떤 모델이야? 출시 시기와 함께 알려주고, 모르면 모른다고 답해줘.
```

실습 2. 에이전트 실습 준비 (4강)

0단계. 설치와 실습 폴더 만들기

터미널(맥: 터미널, 윈도우: Git Bash 권장)에 입력합니다. 설치 방법은 바뀔 수 있으니 공식 문서를 먼저 확인합니다.

```bash
npm install -g @anthropic-ai/claude-code
```

zip을 쓰지 않는 경우 실습 폴더를 직접 만듭니다.

```bash
mkdir agent-lab && cd agent-lab
git init
mkdir -p data output .claude/hooks .claude/skills/weekly-report .claude/agents
```

가짜 데이터 만들기 (파이썬 필요). 아래 내용을 `make_sample_data.py` 로 저장합니다.

python

```python
# 실습용 가짜 데이터 생성기 (실제 개인정보·내부 자료 아님)
# 실행: python make_sample_data.py   (필요: pip install openpyxl)
from openpyxl import Workbook

def save(path, header, rows):
    wb = Workbook(); ws = wb.active; ws.title = "Sheet1"
    ws.append(header)
    for r in rows: ws.append(r)
    wb.save(path)

# 1) 주간 출동 실적 (6강 데모용) - 마지막 열 '합계'에 일부러 틀린 값 1개
rows = [
    ["가센터", 3, 5, 21, 29],
    ["나센터", 2, 4, 18, 24],
    ["다센터", 1, 7, 25, 33],
    ["라센터", 4, 3, 15, 22],   # 정답 22
    ["마센터", 2, 6, 19, 30],   # 오류: 정답은 27
]
save("data/weekly_stats.xlsx", ["센터", "화재", "구조", "구급", "합계"], rows)

# 2) 예산 집행 현황 (4강 지시 비교용)
budget = [
    ["소방장비 유지관리", 50000, 32000],
    ["청사 시설관리", 80000, 61000],
    ["안전교육 운영", 12000, 9000],
    ["의용소방대 지원", 30000, 15000],
    ["119생활안전", 20000, 17500],
]
save("data/budget.xlsx", ["세부사업", "예산액(천원)", "집행액(천원)"], budget)
print("data/weekly_stats.xlsx, data/budget.xlsx 생성 완료")
```

bash

```bash
pip install openpyxl
python make_sample_data.py
```

1단계. 에이전트 실행과 첫 지시

bash

```bash
claude
```

```text
이 폴더에 README.md를 만들고 프로젝트 목적을 "업무 자동화 실습"으로 적어줘
```

2단계. CLAUDE.md (업무 수첩)

세션에서 `/init`을 실행하거나, 아래 내용을 `CLAUDE.md`로 저장합니다.

```markdown
# 프로젝트 규칙
- 언어: 한국어, 문서는 간결체
- 산출물은 output/ 폴더에 저장
- 원본 데이터(data/)는 절대 수정 금지
- 작업 완료 후 변경 파일 목록을 보고할 것

# 자주 쓰는 명령
- 데이터 다시 만들기: python make_sample_data.py
```

3단계. 나쁜 지시와 좋은 지시 비교

나쁜 지시:

```text
보고서 만들어줘
```

`/clear`로 대화를 비운 뒤 좋은 지시:

```text
목표: data/budget.xlsx로 세부사업별 집행률 보고서 1쪽을 만들어줘.
맥락: 과장 결재용이고 금액 단위는 천원이야.
제약: 표 1개, 본문 300자 이내, 원본(data/)은 수정 금지, 결과는 output/에 저장.
완료 기준: 표의 예산액·집행액 합계가 원본과 일치하는지 파이썬으로 검증하고 결과를 보고해.
```

4단계. 계획 모드

`Shift+Tab`으로 계획 모드로 바꾼 뒤:

```text
data 폴더의 엑셀 두 개를 하나의 요약 보고서로 합치려고 해. 실행하지 말고 단계별 계획만 보여줘.
```

설명 3. MCP·스킬·훅 (5강)

MCP 연결

```bash
claude mcp add filesystem -- npx -y @modelcontextprotocol/server-filesystem ./data
claude mcp list
```

세션 안에서 `/mcp`로 상태를 본 뒤:

```text
data 폴더 파일 목록과 각 파일의 첫 줄(제목 행)을 보여줘
```

팀 공유용 `.mcp.json`:

```json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./data"]
    }
  }
}
```

스킬 만들기

`.claude/skills/weekly-report/SKILL.md`로 저장:

```markdown
---
name: weekly-report
description: 주간 업무보고서(주간 출동 실적 보고) 작성 요청 시 사용. data/ 엑셀을 읽어 outpu
---

# 주간 업무보고 작성 절차
1. data/ 폴더의 최신 엑셀을 읽는다
2. 항목별 건수와 합계를 계산한다
3. output/주간보고_YYYYMMDD.md로 저장한다
   - 제목 / 주요 실적(3줄) / 수치 표 / 다음 주 계획
4. 표 합계가 원본과 일치하는지 검증하고 결과를 보고한다
```

새 세션에서 스킬 호출 확인:

```text
이번 주 주간보고 만들어줘
```

훅 만들기

`.claude/hooks/protect-data.sh` 로 저장:

```bash
#!/bin/bash
# 표준입력으로 받은 도구 정보에서 명령어를 꺼낸다
cmd=$(jq -r '.tool_input.command // ""')
if echo "$cmd" | grep -q 'data/' && echo "$cmd" | grep -qE '(^|**:space:**)(rm|mv)*
  echo "data/ 원본은 삭제·이동할 수 없습니다" >&2
  exit 2
fi
exit 0
```

`.claude/settings.json` 으로 저장 (권한 + 훅):

```json
{
  "permissions": {
    "allow": ["Read", "Bash(python:*)", "Bash(git status)"],
    "deny": ["Bash(rm -rf:*)", "Read(./.env)"]
  },
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "hooks": [ { "type": "command", "command": "bash .claude/hooks/protect-data
    ],
    "PostToolUse": [
      { "matcher": "Edit|Write",
        "hooks": [ { "type": "command",
          "command": "jq -r '.tool_input.file_path' >> output/change.log" } ] }
    ]
  }
}
```

훅 스크립트를 실행 가능하게 하고, 에이전트 없이 먼저 직접 시험합니다 (jq 설치 필요).

```bash
chmod +x .claude/hooks/protect-data.sh
echo '{"tool_input":{"command":"rm data/budget.xlsx"}}' | bash .claude/hooks/protec
echo '{"tool_input":{"command":"ls data/"}}' | bash .claude/hooks/protect-data.sh;
```

첫 줄은 `exit=2`(차단), 둘째 줄은 `exit=0`(통과)이 나오면 정상입니다. 세션을 다시 시작한 뒤 에이전트에게 시험합니다.

```text
data 폴더의 budget.xlsx를 지워줘
```

```text
output 폴더에 test.md를 만들고 "훅 테스트"라고 적어줘
```

`output/change.log`에 test.md 경로가 남으면 성공입니다.

설명 4. 멀티에이전트 데모 (6강)

`.claude/agents/analyst.md`:

```markdown
---
name: analyst
description: 엑셀·CSV 데이터 분석 전담. 수치 계산과 표 작성이 필요할 때 사용.
tools: Read, Bash, Write
---
너는 데이터 분석가다. 계산은 반드시 파이썬으로 실행해 검증하고,
결과는 output/analysis.md에 표로 저장한다. 해석은 하지 않는다.
```

`.claude/agents/reviewer.md`:

```markdown
---
name: reviewer
description: 산출물 검토 전담. 보고서 완성 후 수치·형식 오류 확인 시 사용.
tools: Read, Bash
---
너는 검토자다. 원본 데이터와 보고서 수치를 대조하고
오류 목록만 보고한다. 파일을 직접 수정하지 않는다.
```

세션에서 `/agents`로 두 에이전트가 보이는지 확인한 뒤 한 번에 지시합니다.

```text
1) analyst 서브에이전트로 data/weekly_stats.xlsx를 분석하고
2) weekly-report 스킬로 보고서를 작성한 뒤
3) reviewer 서브에이전트로 검토해서
오류가 있으면 고친 최종본을 output/에 저장해줘
```

성공 확인: reviewer가 "마센터 합계 30, 계산값 27" 오류를 보고하고, 최종본에는 27로 고쳐져 있으면 성공입니다.

심화(선택) - 작업 폴더를 나눠 에이전트 두 개를 동시에:

```bash
git worktree add ../agent-lab-b -b feature-b
```

설명 5. 활용편 지시문

판정 요청:

```text
아래 업무가 AI에게 맡길 만한지 판정해줘.
기준: 1) 개인정보·비공개 자료가 들어가는가 2) 결과를 사람이 검증할 수 있는가 3) 주 1회 이상 같은
결과를 수준 1(대화형 AI), 수준 2(스킬), 수준 3(파이프라인) 중 하나로 답하고 이유를 3줄로 써줘.
업무: (여기에 내 업무를 한 문장으로)
```

스킬 초안 요청:

```text
아래 업무 절차를 Claude Code 스킬(SKILL.md)로 만들어줘.
description은 "○○ 요청 시 사용"처럼 언제 쓰는지 구체적으로 쓰고, 마지막 단계에는 결과 검증을 넣
절차: (여기에 내 업무 순서를 번호로)
```

검토 에이전트 요청:

```text
읽기 전용 reviewer 서브에이전트를 만들어줘. tools는 Read와 Bash만 주고, 원본과 결과를 대조해
```

원본 보호 훅 요청:

```text
data/ 폴더의 파일을 삭제·이동하려 하면 막는 PreToolUse 훅을 만들고, 에이전트 없이 직접 시험하는
```